Iteration:

A loop statement allows us to execute a statement or group of statements multiple times as long as the condition is true. Repeated execution of a set of statements with the help of loops is called iteration.
Loops statements are used when we need to run same code again and again, each time with a different value.

Statements:

In Python Iteration (Loops) statements are of three types:
1. While Loop
2. For Loop
3. Nested For Loops

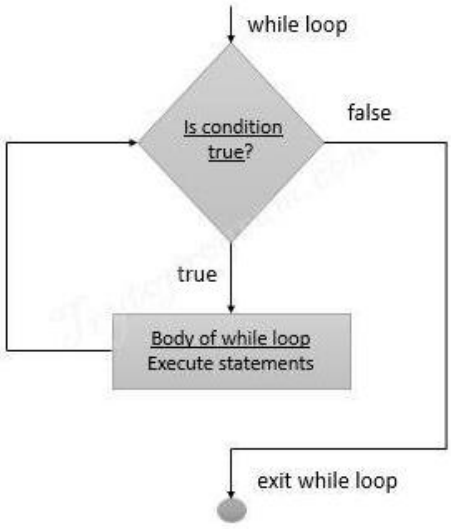
While loop:

- Loops are either infinite or conditional. Python while loop keeps reiterating a block of code defined inside it until the desired condition is met.
- The while loop contains a boolean expression and the code inside the loop is repeatedly executed as long as the boolean expression is true.
- The statements that are executed inside while can be a single line of code or a block of multiple statements.

Syntax:

```
while(expression):
        Statement(s)
```

Flowchart:



Example Programs:

1. ---

```
i=1
while i<=6:
        print("Mrcet college")
        i=i+1
```

output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh1.py
Mrcet college
Mrcet college
Mrcet college
Mrcet college
Mrcet college
Mrcet college
```

2. ---

```
i=1

while i<=3:
   print("MRCET",end="  ")
   j=1
   while j<=1:
      print("CSE DEPT",end="")
      j=j+1
   i=i+1
   print()
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh2.py

MRCET  CSE DEPT

MRCET  CSE DEPT

MRCET  CSE DEPT
```

3. ---

```
   i=1
```

```
j=1
while i<=3:
    print("MRCET",end="  ")

    while j<=1:
        print("CSE DEPT",end="")
        j=j+1
    i=i+1
    print()
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh3.py

MRCET  CSE DEPT
MRCET
MRCET
```

4. ---

```
i = 1
while (i < 10):
      print (i)
      i = i+1
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh4.py
1
2
3
4
5
6
7
8
9
```

2. ---

```
a = 1
b = 1
while (a<10):
    print ('Iteration',a)
    a = a + 1
    b = b + 1
```

```
        if (b == 4):
            break
    print ('While loop terminated')
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh5.py
Iteration 1
Iteration 2
Iteration 3
While loop terminated
-------------------------
count = 0
while (count < 9):
   print("The count is:", count)
   count = count + 1
print("Good bye!")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh.py =
The count is: 0
The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5
The count is: 6
The count is: 7
The count is: 8
Good bye!
```

For loop:

Python **for loop** is used for repeated execution of a group of statements for the desired number of times. It iterates over the items of lists, tuples, strings, the dictionaries and other iterable objects

Syntax: for var in sequence:

Statement(s)

A sequence of values assigned to var in each iteration

Holds the value of item in sequence in each iteration

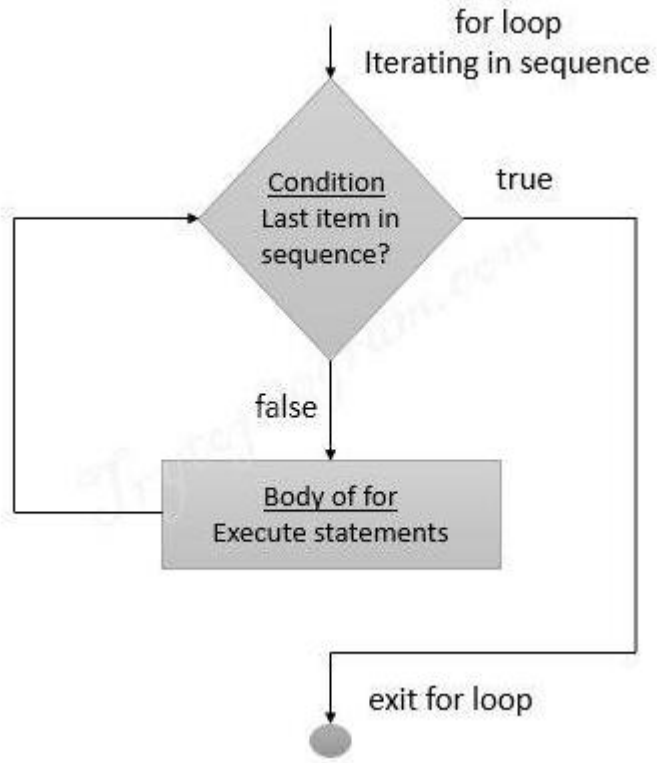
Sample Program:

```
numbers = [1, 2, 4, 6, 11, 20]
seq=0
for val in numbers:
    seq=val*val
    print(seq)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/fr.py
1
4
16
36
121
400
```

Flowchart:



Iterating over a list:

```
#list of items
list = ['M','R','C','E','T']
i = 1

#Iterating over the list
for item in list:
  print ('college ',i,' is ',item)
  i = i+1
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/lis.py
college  1  is  M
college  2  is  R
college  3  is  C
college  4  is  E
college  5  is  T
```

Iterating over a Tuple:

```
tuple = (2,3,5,7)
print ('These are the first four prime numbers ')
#Iterating over the tuple
for a in tuple:
  print (a)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fr3.py
These are the first four prime numbers
2
3
5
7
```

Iterating over a dictionary:

```
#creating a dictionary
college = {"ces":"block1","it":"block2","ece":"block3"}

#Iterating over the dictionary to print keys
print ('Keys are:')
```

```
for keys in college:
  print (keys)

#Iterating over the dictionary to print values
print ('Values are:')
for blocks in college.values():
  print(blocks)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/dic.py
Keys are:
ces
it
ece
Values are:
block1
block2
block3
```

Iterating over a String:

```
#declare a string to iterate over
college = 'MRCET'

#Iterating over the string
for name in college:
  print (name)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/strr.py
M
R
C
E
T
```

Nested For loop:

When one Loop defined within another Loop is called Nested Loops.

Syntax:

```
for val in sequence:
        for val in sequence:
```

statements

statements

 # Example 1 of Nested For Loops (Pattern Programs)

```
for i in range(1,6):
  for j in range(0,i):
    print(i, end=" ")
  print('')
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/nesforr.py
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

 # Example 2 of Nested For Loops (Pattern Programs)

```
for i in range(1,6):
  for j in range(5,i-1,-1):
    print(i, end=" ")
  print('')
```

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/nesforr.py
```

Output:

```
1 1 1 1 1
2 2 2 2
3 3 3
4 4
```

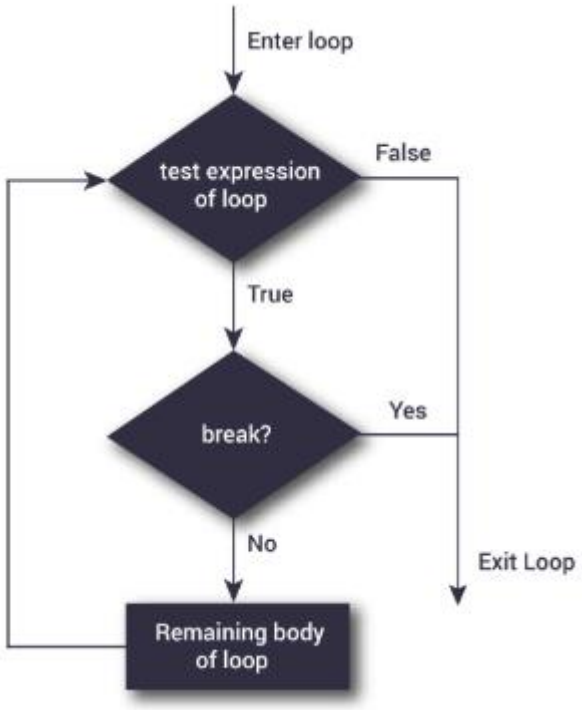
Break and continue:

In Python, **break and continue** statements can alter the flow of a normal loop. Sometimes we wish to terminate the current iteration or even the whole loop without checking test expression. The break and continue statements are used in these cases.

Break:

The break statement terminates the loop containing it and control of the program flows to the statement immediately after the body of the loop. If break statement is inside a nested loop (loop inside another loop), break will terminate the innermost loop.

Flowchart:



The following shows the working of break statement in for and while loop:

```
for var in sequence:
        # code inside for loop
        If condition:
                break (if break condition satisfies it jumps to outside loop)
        # code inside for loop
# code outside for loop
```

```
while test expression

        # code inside while loop
        If condition:
                break (if break condition satisfies it jumps to outside loop)
        # code inside while loop
# code outside while loop
```

Example:

```
for val in "MRCET COLLEGE":
   if val == " ":
      break
   print(val)

print("The end")
```

Output:

```
M
R
C
E
T
The end
```

 # Program to display all the elements before number 88

```
for num in [11, 9, 88, 10, 90, 3, 19]:
  print(num)
  if(num==88):
          print("The number 88 is found")
          print("Terminating the loop")
          break
```

Output:

```
11
9
88
The number 88 is found
```

Terminating the loop

```
#-----------------------------------
for letter in "Python": # First Example
   if letter == "h":
      break
   print("Current Letter :", letter )
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/br.py =

Current Letter : P

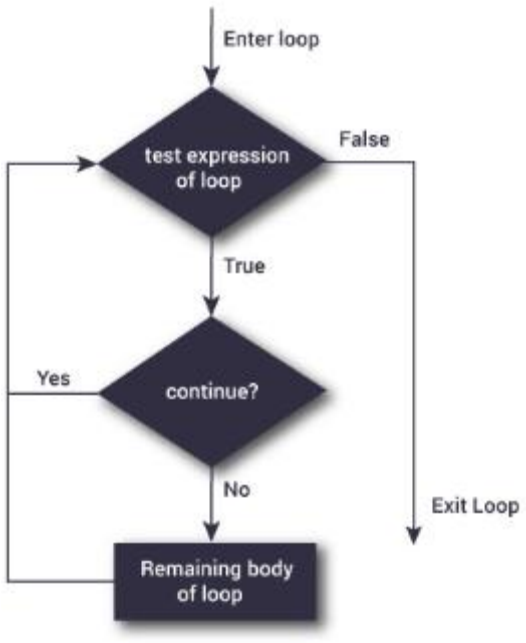
Current Letter : y

Current Letter : t

Continue:

The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

Flowchart:



The following shows the working of break statement in for and while loop:

```
for var in sequence:
        # code inside for loop
        If condition:
                continue (if break condition satisfies it jumps to outside loop)
        # code inside for loop
# code outside for loop


while test expression

        # code inside while loop
        If condition:
                continue(if break condition satisfies it jumps to outside loop)
        # code inside while loop
# code outside while loop
```

Example:

```
# Program to show the use of continue statement inside loops

for val in "string":
   if val == "i":
      continue
   print(val)

print("The end")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/cont.py
s
t
r
n
g
The end

# program to display only odd numbers

for num in [20, 11, 9, 66, 4, 89, 44]:
```

```
    # Skipping the iteration when number is even
    if num%2 == 0:
        continue
    # This statement will be skipped for all even numbers
    print(num)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/cont2.py
11
9
89
```

```
#------------------
for letter in "Python": # First Example
    if letter == "h":
        continue
    print("Current Letter :", letter)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/con1.py
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
```

Pass:

In Python programming, pass is a null statement. The difference between a comment and pass statement in Python is that, while the interpreter ignores a comment entirely, pass is not ignored.
pass is just a placeholder for functionality to be added later.

Example:

```
sequence = {'p', 'a', 's', 's'}
for val in sequence:
    pass
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/f1.y.py
```

```
>>>
```

Similarily we can also write,

```
def f(arg): pass    # a function that does nothing (yet)

class C: pass       # a class with no methods (yet)
```